

Phase I: Context-Aware Specification Synthesis

✓ Successful Patterns

✗ Common Pitfalls

.....
If blind review fails frequently on specific constraints, refine the question template to explicitly **state the priority of conflicting rules** (e.g., "Distance constraints take precedence over color constraints")



Plan

- todo_list
- completed_list
- internal experience
- information

Action

Thought: ... my thought
Code:
```python  
Write(template.txt,  
generator.py, validator.py)

## Interactive Coding & Execution Check

### Input Generator

- Samples input pairs (s, t)
- Controls difficulty and randomness
- Produces structured test cases

### Validator

- Checks logical consistency
- Executes candidate solution
- Returns {pass/fail, error\_type}

### Execution

- Run candidate solution
- Validator returns.
- If fail → feedback to agent

### Error Feedback

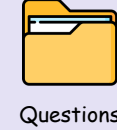
- Runtime exception
- Logical mismatch
- Invalid mapping



## Phase II: Multi-Gate Validation Protocol

### Gate 1: Static Quality Assurance

- Readability
- Completeness
- Difficulty Distribution

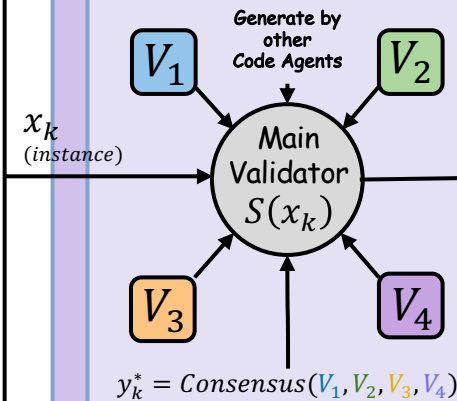


Quality Agent

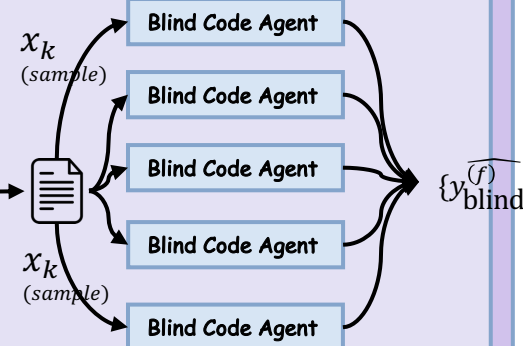
Gate I Pass?

### Gate2: Consensus-based Dynamic Verification

#### Ensemble Ground Truth Generation



#### Code-Augmented Blind Review



$$\text{Pass} \Leftrightarrow \frac{1}{N} \sum \mathbb{I}[\widehat{y_{\text{blind}}^{(f)}} \equiv y_k^*] \geq \tau$$

## Phase III: Debugging & Finalization

### Failure Analysis

- Specification Errors
  - Ambiguity
  - Missing constraints
- Implementation Errors
  - Logic bugs
  - Runtime errors

### Plan Update

- Modify the strategy
- Refine the corner cases
- Update the experience

### Delivery & Finalization

Generator

Validator

Prompt templates

Synthesized data

Agent traces

Experience Update

Consensus Reached?

$\tau$

Syntax OK?

